C++ for Technical Writers

A first look at C++ for people whose job is to explain it — starting from nothing, on a CentOS box, with a small program that actually does something.

N. B. SMITH

Contents

Before we start

A short note about what this is, what it is not, and who it is for.

PART ONE — WHY BOTHER

Why a writer should learn to read code

There is a difference between knowing the name of a thing and knowing the thing. Most of what goes wrong in software documentation lives in that gap.

PART TWO — GETTING A MACHINE TO OBEY YOU

Get a compiler onto your CentOS box

Fifteen minutes of setup, most of which is waiting. You need three things: a compiler, a build tool, and a debugger you will not use for a while.

Make the machine say something

The traditional first program prints a greeting. We will do that, and then we will break it on purpose, which is where the actual learning is.

What those six lines actually were

Every piece of that program is there for a reason. Let us take it apart, because these same pieces are in every file you will ever be asked to document.

PART THREE — BUILDING SOMETHING REAL

Read a file of messages

Now we start the real project. We are building a small tool that reads status messages and makes them legible — the sort of utility that exists on every system you will ever document.

What the compiler is worried about

C++ insists on knowing what kind of thing every value is, before the program runs. This is either an annoyance or the most useful feature in the language, depending on the day.

Pull the message apart

Printing a line is not much use. Now we split each message into its fields, put them in a shape with names, and hand the work to a function.

The ampersands and asterisks, without the fear

This is the part people warn you about. It is not hard. It has simply been explained badly for forty years, usually by people who understood it too well to remember what confused them.

Split it into a real project

One file is a script. Several files with a build recipe is a project — and it is the arrangement you will meet in every codebase you are asked to document.

PART FOUR — TURNING CODE INTO DOCUMENTATION

The header file is your table of contents

If you only ever learn to read one kind of C++ file, make it this one. A header tells you what a component offers the world, which is exactly the set of things a reader can do.

What this does to your working life

The technical payoff is real but modest. The change in how you work with engineers is the large one, and it is worth being explicit about why.

REFERENCE

Glossary

The formal name, and then what it actually is. If the second column is missing from a glossary, the glossary is decoration.

Commands and flags worth remembering

A short list. Everything here appeared somewhere in this tutorial, so you have already run all of it at least once.

Before we start

> *A short note about what this is, what it is not, and who it is for.*

Who this is for

You write documentation. You work near software engineers. You have never written a line of C++ and you have started to suspect that this is a problem — not because anyone told you so, but because you keep having to take their word for things.

That suspicion is correct, and this is the fix. You will not become a programmer. You will become someone who can open a source file, follow what it is doing, and ask a question that makes an engineer stop and think. That turns out to be enough. It is, honestly, quite a lot.

Everything here runs on CentOS Stream 9. If you have a different Linux, almost all of it works anyway; the package-install command is the part that changes.

Release statement

This document contains none of the following:

— Classified information of any level.

— Controlled Unclassified Information (CUI).

— Proprietary information of any company, or information subject to a non-disclosure agreement.

— Export-controlled technical data under the International Traffic in Arms Regulations (ITAR), 22 CFR 120–130.

— Export-controlled technical data under the Export Administration Regulations (EAR), 15 CFR 730–774.

All content is original work. The message format used in the example project is invented for teaching and matches no real system. The source code is written for this tutorial and is placed in the public domain. Technical information about the compiler and the operating system comes from publicly released documentation.

This is a technical writing sample. Prepared by Nathan B. Smith.

1 Why a writer should learn to read code

> *There is a difference between knowing the name of a thing and knowing the thing. Most of what goes wrong in software documentation lives in that gap.*

The trouble with names

Suppose an engineer tells you the system has a `MessageDispatcher`. You write it down. You put it in the guide: *The MessageDispatcher routes incoming messages to the appropriate handler.* Everyone nods. The document ships.

Now — what does it actually do?

You do not know. You know its name. You have written a sentence that rearranges its name into a definition, which is a very old trick and a completely empty one. If a reader asks "what happens when two messages arrive at the same instant," your sentence has nothing to say. If the engineer renames the class next month, your sentence is not even wrong; it is just gone.

This is the whole problem, and it is worth being blunt about it: you can produce fluent, professional, well-formatted documentation that contains no information. It happens constantly. It happens because writing about a name is easy and finding out what the thing does is hard.

What changes when you can read the source

Open the file. Find `MessageDispatcher`. You see it keeps a list. You see messages go onto the end of the list and come

off the front. You see there is exactly one thread pulling from that list.

Now you know something. Messages are handled one at a time, in arrival order. That is not a name — that is behavior, and it is the thing your reader needed. And you got it in four minutes without interrupting anybody.

Better still, you now have a real question: *what happens when the list fills up?* You go and ask the engineer that question, and something interesting happens to the conversation. You are no longer a person asking to have things explained. You are a person who read the code and found the edge case. Engineers respond to that differently. They will tell you things they would never think to volunteer.

> **WHY THIS MATTERS**
>
> The value is not that you write the code. The value is that the engineer stops translating for you. Every translation step loses something, and the thing it loses is usually the exact detail your reader needed.

You are the easiest person to fool

Here is the uncomfortable part. When you do not understand something, your mind does not announce it. It quietly substitutes a vaguer statement that *feels* like understanding, and you write that down instead. The vaguer statement is smooth. It passes review. Nobody catches it, because the reviewers are doing the same thing with the same gap.

Reading the source is a way of checking yourself. The compiler does not care how confident you sound. The program either does the thing or it does not. That is a rare

and valuable kind of feedback, and very little else in documentation work provides it.

What you will actually be able to do

By the end of this you will be able to:

- Open a C++ source file and tell what it is broadly doing.
- Find every function a component offers to the rest of the system, and know which ones your reader will ever call.
- Spot the error cases the code handles, which is where most missing documentation lives.
- Notice when the code and the specification disagree, and say so.
- Write a build instruction you have personally run, instead of one somebody dictated to you.

You will not be able to write production software, and you should not try. That is a different job and it takes years. But the gap between "cannot read code at all" and "can read code slowly" is the big one. The gap after that is much smaller than people pretend.

One more thing before we start

You are going to type things and they are going to fail. The compiler will produce an error message forty lines long about a missing semicolon. This is normal. It is normal for everybody, at every level of skill, forever.

The trick is to find it interesting rather than humiliating. An error message is the machine telling you exactly where your idea and reality parted company. There is no other tool in your professional life that does that. Enjoy it while you can.

2

Get a compiler onto your CentOS box

> *Fifteen minutes of setup, most of which is waiting. You need three things: a compiler, a build tool, and a debugger you will not use for a while.*

BEFORE YOU BEGIN

A CentOS Stream 9 machine — a virtual machine is completely fine — and an account that can use `sudo`.

A compiler is a program that reads text you wrote and produces instructions the processor can execute. That is the entire idea. The reason it feels mysterious is that the output is unreadable, so you never get to inspect the thing it made. Take it on faith for now; we will poke at it later.

1. Open a terminal and find out what you already have.

   ```
   g++ --version
   ```

   If that prints a version number, skip ahead. If it says `command not found`, keep going.

2 Install the tools.

```
sudo dnf install -y gcc-c++ make gdb
```

Three packages, three jobs:

- `gcc-c++` — the compiler itself. Turns your text into a program.
- `make` — a tool that remembers which compile commands to run so you do not have to.
- `gdb` — a debugger. It lets you stop a running program and look inside it. You will not need it today, but install it now so it is there the first time you want it.

3 Check that it worked.

```
g++ --version
```

> RESULT
>
> You get a version banner. The exact version does not matter for anything we are doing.

4 Make yourself somewhere to work.

```
mkdir -p ~/cpp/msgpeek
cd ~/cpp/msgpeek
```

WHERE THAT LEAVES YOU

You have a compiler and a place to put things. That is genuinely all the setup there is. People will try to sell you an integrated development environment with nine hundred buttons; you do not need one, and starting without one means you will understand what the buttons would have been doing.

TRY THIS

Run `which g++` and then `ls -l` on the path it prints. You are looking at the compiler as a file on disk. It is just a program, like any other. Nothing about it is special except what it does.

3 Make the machine say something

> *The traditional first program prints a greeting. We will do that, and then we will break it on purpose, which is where the actual learning is.*

BEFORE YOU BEGIN

A compiler, and a directory to work in.

---

1 Create a file called `hello.cpp` containing exactly this:

```
#include <iostream>

int main()
{
    std::cout << "The machine is listening.\n";
    return 0;
}
```

Type it by hand. Do not paste it. Typing it wrong and fixing it is the point of the exercise; pasting it correctly teaches you nothing.

2 Compile it.

```
g++ -Wall -o hello hello.cpp
```

Reading that command backwards: take `hello.cpp`, produce (`-o`) a program called `hello`, and tell me about anything that looks suspicious (`-Wall`, meaning "all warnings"). Always use `-Wall`. It is free and it catches real mistakes.

3 Run it.

```
./hello
```

> RESULT
>
> `The machine is listening.`
>
> The `./` is not decoration. It means "the program in this directory." Without it the shell searches a list of standard locations, none of which is here.

4 Now break it. Delete the semicolon at the end of the `std::cout` line and compile again.

```
g++ -Wall -o hello hello.cpp
```

> RESULT
>
> You get an error mentioning the line number and the word `expected`. Look at what it says carefully. The compiler is not confused about semicolons — it is telling you it reached a place where the grammar of the language required one and found something else.

5 Put the semicolon back. Then break it a different way: change `std::cout` to `std::cowt` and compile.

> RESULT
>
> A completely different error, about a name that does not exist. Two kinds of mistake, two kinds of complaint. You have just learned something real about how compilers work, and you learned it by breaking things rather than by reading about it.

6 Fix it and compile one last time so you end with a working program.

WHERE THAT LEAVES YOU

You have written, compiled, and run a program, and you have seen two distinct failure modes. That is more than most people who "cannot code" have ever done.

TRY THIS

Run `ls -l` and compare the size of `hello.cpp` with the size of `hello`. Your six lines of text became tens of thousands of bytes. Ask yourself where the rest came from. (It came from `iostream` and the machinery underneath it. You asked for a lot more than you thought you did.)

4 What those six lines actually were

> *Every piece of that program is there for a reason. Let us take it apart, because these same pieces are in every file you will ever be asked to document.*

The include line

```
#include <iostream>
```

This says: before you compile my file, go find a file called `iostream` and paste its entire contents here. Literally paste. That is not a metaphor for what it does; it is what it does.

`iostream` contains the descriptions of the input and output machinery. Without it, the compiler has never heard of `std::cout` and cannot check whether you are using it correctly.

Angle brackets mean "look in the system directories." Quotation marks mean "look next to my file first." You will see both, and the difference tells you at a glance whether something came from the project or from outside it. That is a useful thing for a documenter to notice.

The function called main

```
int main()
{
    ...
}
```

Every program has exactly one `main`. It is where execution starts. When you type `./hello`, the operating system finds `main` and begins there.

The word `int` in front says that when `main` finishes it will hand back an integer. The parentheses are the list of things you can hand *in* — empty here. The braces mark the beginning and end of the body.

That pattern — a return type, a name, a parenthesized list, a braced body — is a function, and it is the single most important shape in the language. Learn to see it and half of C++ stops looking like noise.

The double colons

`std::cout` means: the thing called `cout` that lives inside the namespace called `std`. A namespace is a surname. It exists so that two different libraries can both have something called `Message` without a fistfight.

`std` is the standard library — the collection that comes with the language. When you see a name with no namespace in front of it, somebody in the project wrote it. When you see `std::`, they did not. For a writer trying to work out what a project is responsible for, that distinction is worth a great deal.

The arrows

```
std::cout << "The machine is listening.\n";
```

Read the arrows as "send this into that." The text flows into `cout`, which is the standard output stream, which for you means the terminal window.

Here is the part that is genuinely peculiar, and I want to flag it rather than let you assume you missed something: those arrows are not built into the language as an output operator. They are the bit-shift operator, and somebody redefined what it means when the left-hand side is an output stream. C++ lets you do that. It is called operator overloading, and it explains a great deal of the strangeness you will meet later.

Is that a good design decision? People have argued about it for thirty-five years. You do not have to have an opinion. You just have to know that the same symbol can mean different things depending on what is on either side of it — because when you are reading unfamiliar code, that is the sort of thing that makes you think you have gone mad.

The `\n` is a newline character, written with two characters because there is no way to type an invisible one.

The return

```
return 0;
```

Zero means "nothing went wrong." Any other number means something did. This is backwards from what you would guess, and there is a reason: there is only one way to succeed, and many ways to fail, so the interesting numbers are reserved for the failures.

Every shell script that checks whether a command worked is reading this number. When you document a command-line tool and the guide has a table of exit codes, this is where those codes come from.

> **WHY THIS MATTERS**
>
> You now know where exit codes live in the source. Next time an engineer says "it returns non-zero on failure," you can open the file, find every `return` in `main`, and write the actual table. Nobody will have to remember it for you.

The semicolons and braces

A semicolon ends a statement. A brace groups statements. The language does not care about your line breaks or your indentation at all — you could write the whole program on one line and it would compile identically.

Which raises the question: why bother indenting? Because the compiler is not the audience. The next person to read it is, and that person is frequently you, eight months later, having forgotten everything. This is exactly the argument you make about documentation, and it lands the same way.

5 Read a file of messages

> *Now we start the real project. We are building a small tool that reads status messages and makes them legible — the sort of utility that exists on every system you will ever document.*

Here is our message format. It is invented for this tutorial, but it has the shape of a thousand real ones: fields in a fixed order, separated by a delimiter, with a blob of loosely structured detail at the end.

```
MSG|0042|2026-07-25T14:32:10Z|SENSOR-ALPHA|STATUS|OK|
battery=94;temp=31.2
```

Seven fields: a literal tag, a sequence number, a timestamp, the source, the message type, a state, and the details. Our tool — `msgpeek` — will read a file of these and print them in a form a human can scan.

We are going to build it in three passes, and each pass will work before we go on to the next. That habit is worth more than any individual technique in this document.

1. Make a file of test messages called `traffic.txt`.

   ```
   MSG|0042|2026-07-25T14:32:10Z|SENSOR-ALPHA|STATUS|OK|
   battery=94;temp=31.2
   MSG|0043|2026-07-25T14:32:41Z|SENSOR-BRAVO|STATUS|
   DEGRADED|battery=11;temp=44.9
   MSG|0044|2026-07-25T14:33:02Z|SENSOR-ALPHA|ALERT|
   FAULT|code=E17;retries=3
   MSG|0045|2026-07-25T14:33:15Z|GATEWAY-01|STATUS|OK|
   uptime=88213
   ```

2 Write `msgpeek.cpp`.

```
#include <iostream>
#include <fstream>
#include <string>

int main(int argc, char* argv[])
{
    if (argc < 2) {
        std::cerr << "usage: msgpeek <file>\n";
        return 1;
    }

    std::ifstream in(argv[1]);
    if (!in) {
        std::cerr << "cannot open " << argv[1] <<
"\n";
        return 2;
    }

    std::string line;
    int count = 0;

    while (std::getline(in, line)) {
        ++count;
        std::cout << count << ": " << line << "\n";
    }

    std::cout << "read " << count << " messages\n";
    return 0;
}
```

3 Compile and run it.

```
g++ -Wall -o msgpeek msgpeek.cpp
./msgpeek traffic.txt
```

RESULT

Four numbered lines and a count.

4 Now run it wrong, on purpose, twice.

```
./msgpeek
./msgpeek nosuchfile.txt
echo $?
```

`echo $?` prints the exit code of the last command. You will see the 1 and the 2 you wrote into the source appearing in the shell. That is the whole mechanism, end to end.

WHERE THAT LEAVES YOU

A working tool, forty lines long, with two documented failure modes.

The three new ideas in that file

Arguments to main. `argc` is how many words were on the command line, and `argv` is the words themselves. The program's own name is `argv[0]`, which is why we check for `argc < 2` rather than `argc < 1`. Counting from zero will trip you at least once. Everyone.

cerr as well as cout. Errors go to `std::cerr`, a separate stream, so that someone redirecting the normal output to a file still sees the complaint on screen. When you document a tool's output, this distinction is worth mentioning; most guides never do.

The while loop. `std::getline` tries to read a line. It hands back something that counts as true if it worked and false if it did not. So the loop means: keep reading until reading stops working. It is one line and it handles the end of the file, which is neat once you see it.

TRY THIS

Feed it a file that does not end with a newline. Then feed it an empty file. Then feed it a directory. Three experiments, three behaviors — and you have just written the "Error handling" section of the user guide from observation instead of from hope.

6

What the compiler is worried about

> *C++ insists on knowing what kind of thing every value is, before the program runs. This is either an annoyance or the most useful feature in the language, depending on the day.*

The idea

In your file you wrote `int count = 0;` and `std::string line;`. You told the compiler: this thing is a whole number, that thing is a piece of text. From then on it holds you to it. Try to divide the text by two and it refuses to build the program at all.

Some languages let you do whatever you like and fall over at three in the morning in production. C++ would rather fail on your desk at eleven in the morning. That trade is the reason it is still used for systems that are not allowed to stop.

The ones you will actually meet

TYPES YOU WILL SEE CONSTANTLY

Written as	What it holds	What it actually is, plainly
`int`	A whole number	Counts, indexes, sizes, error codes. No fractions.
`double`	A number with a fractional part	Temperatures, voltages. Slightly imprecise, always.
`bool`	True or false	A switch. Flags, checks, and the result of comparisons.
`char`	One character	A single letter. Really a small number wearing a hat.
`std::string`	Text of any length	What you think of as a word or a line. Grows as needed.
`std::vector<T>`	A list of some type T	A row of boxes, all the same kind, numbered from zero.
`size_t`	A whole number, never negative	Used for sizes and positions. Appears everywhere.
`void`	Nothing	A function marked void hands nothing back.

The one that will bite you

`double` is not exact. Write a program that adds one tenth to itself ten times and compares the answer to one, and the comparison fails. This is not a bug in C++; it is a consequence of storing decimal fractions in binary, the same way one third is untidy in decimal.

You do not need to understand the floating-point standard. You need to know that comparing two decimals for exact equality is a mistake, so that when you see code doing it, you have found something worth asking about. That is the

level of understanding this document is aiming at throughout: not enough to write it, enough to smell it.

> **TRY THIS**
>
> Write a five-line program that prints `0.1 + 0.2 == 0.3`. Then print `0.1 + 0.2` with fifteen digits of precision. The answer is one of the small genuine surprises in computing, and it is much better discovered than described.

Why this matters to your day job

When you document a field in a message, the type in the source tells you things the specification often forgets: whether a value can be negative, whether it can have a decimal part, how large it is allowed to get before it wraps around.

A sequence number stored as a 16-bit integer rolls over at 65,535. If nobody documented that, it is because nobody looked at the type. You can look at the type.

7

Pull the message apart

> *Printing a line is not much use. Now we split each message into its fields, put them in a shape with names, and hand the work to a function.*

Two new ideas here, and they are the two that matter most for reading real code: the **struct**, which is a bundle of named values, and the **function**, which is a piece of work you can call by name.

A struct is exactly the thing you have been drawing on whiteboards for years. A message has a sequence number, a

timestamp, a source. Instead of three loose variables, one thing with three named parts. That is all it is.

1 Replace `msgpeek.cpp` with this.

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>

struct Message {
    std::string tag;
    std::string sequence;
    std::string timestamp;
    std::string source;
    std::string type;
    std::string state;
    std::string details;
    bool        valid = false;
};

std::vector<std::string> split(const std::string&
text, char delimiter)
{
    std::vector<std::string> fields;
    std::string current;

    for (char c : text) {
        if (c == delimiter) {
            fields.push_back(current);
            current.clear();
        } else {
            current += c;
        }
    }
    fields.push_back(current);
    return fields;
}

Message parse(const std::string& line)
{
    Message m;
    std::vector<std::string> f = split(line, '|');

    if (f.size() != 7) {
        return m;              // valid stays false
    }

    m.tag       = f[0];
    m.sequence  = f[1];
    m.timestamp = f[2];
    m.source    = f[3];
    m.type      = f[4];
    m.state     = f[5];
    m.details   = f[6];
```

```
    m.valid     = true;
    return m;
}

int main(int argc, char* argv[])
{
    if (argc < 2) {
        std::cerr << "usage: msgpeek <file>\n";
        return 1;
    }

    std::ifstream in(argv[1]);
    if (!in) {
        std::cerr << "cannot open " << argv[1] <<
"\n";
        return 2;
    }

    std::string line;
    int good = 0, bad = 0;

    while (std::getline(in, line)) {
        if (line.empty()) continue;

        Message m = parse(line);
        if (!m.valid) {
            ++bad;
            std::cerr << "malformed: " << line <<
"\n";
            continue;
        }

        ++good;
        std::cout << "[" << m.sequence << "] "
                  << m.source << " " << m.type
                  << " -> " << m.state
                  << "  (" << m.details << ")\n";
    }

    std::cout << good << " good, " << bad << "
malformed\n";
    return bad > 0 ? 3 : 0;
}
```

2 Compile and run.

```
g++ -Wall -o msgpeek msgpeek.cpp
./msgpeek traffic.txt
```

3 Add a deliberately broken line to `traffic.txt` and run it again.

```
MSG|0046|SENSOR-CHARLIE|STATUS
```

> RESULT
>
> One complaint on the error stream, the count updated, and an exit code of 3. Check it with `echo $?`.

What to notice

The function signature is the contract. Look at this line:

```
Message parse(const std::string& line)
```

Read it as a sentence: *give me a line of text, and I will give you back a Message.* That is everything a caller needs to know. It is also, almost word for word, the documentation for that function. When you learn to read signatures, half your reference material writes itself.

The word const is a promise. It says this function will not modify the text you handed it. When you are documenting whether a call has side effects, `const` is the first place to look, and it does not lie — the compiler enforces it.

The validity flag is a design decision. Notice that `parse` does not crash or complain about a bad line; it hands back a Message with `valid` set to false and lets the caller decide. That is a choice, and different engineers make it differently. Spotting choices like this in code is what lets you ask the

question that improves the software rather than just describing it.

The loop with the colon. `for (char c : text)` means "for each character in text." It reads almost like English, and modern C++ has a lot of this. If you learned that C++ was unreadable, you learned it from code written before about 2011.

> **WHY THIS MATTERS**
>
> You have now met the three shapes that make up nearly all production C++: a struct that holds data, a function that transforms it, and a loop that applies the function to many things. Large systems are these three shapes repeated at scale with better names.

8 The ampersands and asterisks, without the fear

> *This is the part people warn you about. It is not hard. It has simply been explained badly for forty years, usually by people who understood it too well to remember what confused them.*

Start with the problem, not the syntax

You have a message. You want to hand it to a function. There are two things the machine could do: copy the whole message and give the function the copy, or tell the function where the original lives and let it go and look.

That is the entire concept. Copy, or directions. Everything else is notation.

Copying is safe — the function cannot damage your original — but it costs time and memory, and if the thing is large you notice. Directions cost nothing, but now the function can change your original, which is sometimes exactly what you want and sometimes a catastrophe.

The notation

THREE WAYS TO PASS SOMETHING TO A FUNCTION

Written as	Means	What you can conclude
`Message m`	Give me a copy	Your original is untouchable. Costs a copy.
`const Message& m`	Show me yours; I promise not to touch it	Fast and safe. By far the most common in good code.
`Message& m`	Show me yours; I may change it	This function has a side effect. Document it.

So when you saw `const std::string& line` in the parser, you were reading a promise: fast, and your string comes back unharmed.

The asterisk

A pointer, written with `*`, is the older and rawer version of the same idea: a variable that holds an address. The important practical difference is that a pointer is allowed to point at nothing at all, and a reference is not.

Pointing at nothing is called being null, and using a null pointer is the classic way for a C++ program to die on the spot. When you see code checking `if (p != nullptr)`, that is somebody being careful. When you see a pointer used without a check, you have found a question worth asking — politely, and in person.

> **WHY THIS MATTERS**
>
> Whether a function copies, borrows, or borrows-and-modifies is not a stylistic detail. It determines whether a caller's data can change underneath them, which is precisely the sort of thing your reader needs to know and the sort of thing that never makes it into a specification.

Why the language has both

Because it is old, and because it grew. Pointers came from C in the early seventies, when the machine and the programmer were on first-name terms. References were added later to give the same speed with fewer ways to hurt yourself.

You will meet both in any real codebase, often in the same file, because code accumulates in layers like sediment. That is not a failure of discipline; it is what happens to anything that stays useful for fifty years. The documentation for such a system has the same geology, which is worth remembering when you are tempted to despair at it.

What you can safely not learn today

There is a great deal more: smart pointers, ownership, move semantics, references to references. It is genuinely interesting and you do not need any of it to read a file and understand what it does.

Resist the urge to learn everything before doing anything. The useful order is the opposite: read some real code, hit something you do not recognize, look up that one thing. Curiosity driven by a specific obstacle sticks. Curiosity driven by a syllabus does not.

9 Split it into a real project

> *One file is a script. Several files with a build recipe is a project — and it is the arrangement you will meet in every codebase you are asked to document.*

We are going to cut the program into three pieces: a header that announces what exists, a source file that implements it, and a main program that uses it. Then a `Makefile` so one command rebuilds whatever changed.

Pay attention to the header. It is about to become the most valuable file in the project as far as you are concerned.

1. Create `message.h`.

```
#ifndef MESSAGE_H
#define MESSAGE_H

#include <string>
#include <vector>

/// One status message, already broken into fields.
struct Message {
    std::string tag;
    std::string sequence;
    std::string timestamp;
    std::string source;
    std::string type;
    std::string state;
    std::string details;
    bool        valid = false;
};

/// Split text on a single delimiter.
/// Always returns at least one field, even for empty
input.
std::vector<std::string> split(const std::string&
text, char delimiter);

/// Parse one line into a Message.
/// On a line without exactly seven fields, returns a
Message
/// whose valid member is false. Never throws.
Message parse(const std::string& line);

#endif
```

2. Create `message.cpp` containing the bodies of `split` and `parse` from the previous version, with `#include "message.h"` at the top and the struct definition removed.

3. Create `main.cpp` containing the `main` function from the previous version, with `#include "message.h"` at the top.

4 Create `Makefile`. The indentation must be real tab characters, not spaces — this is the single most annoying rule in the whole toolchain.

```
CXX      = g++
CXXFLAGS = -Wall -Wextra -std=c++17 -g

msgpeek: main.o message.o
        $(CXX) $(CXXFLAGS) -o msgpeek main.o message.o

main.o: main.cpp message.h
        $(CXX) $(CXXFLAGS) -c main.cpp

message.o: message.cpp message.h
        $(CXX) $(CXXFLAGS) -c message.cpp

clean:
        rm -f msgpeek *.o
```

5 Build it.

```
make
```

RESULT

Three commands run. You did not type them.

6 Run `make` again without changing anything.

RESULT

`make: 'msgpeek' is up to date.` Nothing rebuilt, because nothing changed. That is the whole point of the tool.

7 Touch the header and build again.

```
touch message.h
make
```

> RESULT
>
> Both source files recompile, because the Makefile says both depend on the header. Change only `main.cpp` and watch just that one rebuild. The dependency list is doing real work.

WHERE THAT LEAVES YOU

A three-file project with a build recipe — the ordinary shape of real software.

What the header guard is doing

Those three lines around the file:

```
#ifndef MESSAGE_H
#define MESSAGE_H
...
#endif
```

Remember that `#include` literally pastes. If two files each include this header, and one of them includes the other, the contents arrive twice and the compiler complains that `Message` is defined two times.

The guard says: if this label is not yet set, set it and carry on; otherwise skip to the end. The second paste does nothing. It is a slightly embarrassing solution to a slightly embarrassing problem, and every C++ header in the world has it.

TRY THIS

Delete the three guard lines and run `make`. Read the error. Put them back. Ten seconds of work for a piece of knowledge that will otherwise take you a year to acquire by accident.

10

The header file is your table of contents

> *If you only ever learn to read one kind of C++ file, make it this one. A header tells you what a component offers the world, which is exactly the set of things a reader can do.*

The division of labor

A header says what exists. A source file says how it works. The distinction is not bureaucratic — it maps almost perfectly onto the distinction between reference documentation and internals.

Everything a caller can use appears in the header. Anything not in the header is private machinery, and your reader will never touch it. So the header is a first draft of your reference section, written by the engineer, for free, and kept current because the compiler will not let it drift.

> **WHY THIS MATTERS**
>
> Ask an engineer "what does this component do" and you get a paragraph shaped by whatever they were working on that morning. Read the header and you get the complete list, in their own terms, with nothing left out. Then ask your questions about the parts that puzzle you. The conversation is ten times better.

How to read one, in order

1. **The includes.** What this component depends on. Angle brackets are outside libraries; quotes are project files. A

header that includes fifteen project files is doing too much, and that is worth noticing.

2. **The structs and classes.** The nouns of the system. Their member names are the vocabulary your document should use — and if the code and the specification use different words for the same thing, you have found a real problem and you should raise it.
3. **The function declarations.** The verbs. Each one is a thing a caller can do.
4. **The comments.** Whatever the engineer thought was worth saying. Treat these as a lead, not as truth; comments are not checked by anything and they rot.

Reading our own header as a stranger would

```
Message parse(const std::string& line);
```

What does this one line tell you, before reading any comment?

— It takes a line of text and produces a Message. That is the summary sentence of your reference entry.

— `const` and `&` mean it does not modify or copy your input. No side effects on the caller's data.

— It returns a Message by value, so the caller owns the result and does not have to release anything.

— It has no error parameter and no exception specification, so failure must be reported inside the returned Message — which sends you to look at the `valid` member.

Four facts from one line, and every one of them belongs in the documentation. None of them required understanding the implementation.

The questions a header cannot answer

Be equally clear about the limits. The header will not tell you:

— **Why.** Why seven fields? Why is a malformed line tolerated rather than rejected outright? Intent lives in people, not in code.

— **How fast.** Performance is measured, not declared.

— **Whether it is safe to call from two threads at once.** Occasionally documented, usually not, always important.

— **What the caller is supposed to do about a failure.** The code reports; the operator has to act.

These four are the good questions. They are what you bring to the engineer after you have read the header, and they are the reason a writer who reads code is worth more than a writer who does not — not because they need fewer answers, but because they ask better questions.

11

What this does to your working life

> *The technical payoff is real but modest. The change in how you work with engineers is the large one, and it is worth being explicit about why.*

The old conversation

You ask an engineer to explain a component. They begin at a level they guess is right for you, discover it is wrong, adjust, lose the thread, and finish with something true but incomplete. You write it up. They review it and say "not quite," and cannot easily say why. Two more rounds. Everyone is tired and the document is mediocre.

Nobody is at fault here. Explaining a system to someone who cannot read it is genuinely hard, and engineers are not trained for it. The cost is paid in review cycles, which is the most expensive currency on a documentation project.

The new conversation

You read the header first. You arrive with: "I see parse returns a Message with a valid flag rather than throwing. Was that so the caller could keep going after a bad line, or is there history there?"

Three things just happened. You demonstrated that their time will not be spent on basics. You asked about intent, which is the only thing they actually have that you cannot get elsewhere. And you gave them a specific thing to react to, which is far easier than generating an explanation from nothing.

They will answer that question, and then they will keep going, and the part they volunteer after answering is usually the most valuable paragraph in the eventual document.

Questions worth carrying with you

- What happens when this is called with input nobody expected?
- What does the caller have to do about each failure?
- Can two of these run at once?
- What is the largest value this field can hold before something goes wrong?
- What did this used to do, and why did it change?
- What part of this do you expect people to get wrong?

That last one is the best question in technical writing. Ask it every time. The answer is your troubleshooting section, and no specification anywhere contains it.

A word on staying honest

There is a failure mode on the other side of this, and it is worth naming. A writer who has learned a little code sometimes starts guessing — filling a gap with a plausible inference from the source rather than asking, because asking feels like admitting the limit of what they learned.

Do not do that. Reading code tells you what the program does. It does not tell you what the program is supposed to do, and where those two differ you have found a bug, not a fact to document. Undocumented behavior is not documented behavior. When you are not sure, the honest sentence is "I do not know yet," and it costs you nothing.

> **WHY THIS MATTERS**
>
> The reason to learn this is not to need engineers less. It is to make the time you spend with them worth more — to both of you. That is the whole return on the afternoon you just spent.

Where to go next

Do not buy a nine-hundred-page book. Do this instead: find a small real file in a project you document, open it, and read it until you hit something you do not recognize. Look up that one thing. Close the file. Tomorrow, do it again.

Twenty minutes a day for a month and you will read most of what crosses your desk. The pleasure of it sneaks up on you — there is a particular satisfaction in opening a file that used to be noise and finding that it has become a sentence. That satisfaction is the thing that keeps people learning for forty years, and it is available to you now.

12

Glossary

> *The formal name, and then what it actually is. If the second column is missing from a glossary, the glossary is decoration.*

Term	What it actually is
Argument	A value you hand to a function when you call it.
Build	The whole process of turning source files into a runnable program.
Class	A struct that also carries the functions that work on it, and can hide some of its parts.
Compile	Turn one source file into machine instructions. Happens per file.
const	A promise, enforced by the compiler, that something will not be changed.
Declaration	Announcing that something exists, without saying how it works. Lives in headers.
Definition	Saying how it works. Lives in source files.
Header	A file listing what a component offers. Pasted into other files by #include.
Header guard	Three lines that stop a header being pasted in twice.
Link	Stitch the separately compiled pieces into one program. The step after compiling.
Makefile	A recipe listing what depends on what, so only what changed gets rebuilt.
Namespace	A surname for names, so two libraries can both have a Message.
Null pointer	A pointer aimed at nothing. Using one usually kills the program instantly.
Object file	The compiled form of one source file, ending in .o. Not yet runnable.
Pointer	A variable holding an address — directions to a value rather than the value.
Reference	

Term	What it actually is
	Like a pointer, but it can never point at nothing. Written with an ampersand.
Return type	The kind of thing a function hands back. The first word of its signature.
Signature	A function's name, its parameters, and its return type. Its contract.
Standard library	What comes with the language. Everything with std:: in front of it.
struct	A bundle of named values treated as one thing.
Type	What kind of thing a value is. Checked before the program ever runs.
Vector	A list that grows as needed. Numbered from zero, not one.
Warning	The compiler saying "this is legal but I doubt you meant it." Read them.

13

Commands and flags worth remembering

> *A short list. Everything here appeared somewhere in this tutorial, so you have already run all of it at least once.*

Setting up on CentOS

Command	What it does
`sudo dnf install gcc-c++ make gdb`	Install the compiler, the build tool, and the debugger.
`g++ --version`	Confirm the compiler is present.
`which g++`	Show where the compiler lives on disk.

Compiling

Flag or command	What it does
`g++ -o name file.cpp`	Compile and link into a program called name.
`-Wall`	Turn on the ordinary warnings. Always use it.
`-Wextra`	Turn on more warnings. Use it too.
`-std=c++17`	Ask for a specific version of the language.
`-g`	Include information the debugger needs.
`-c`	Compile only; do not link. Produces a .o file.
`-I dir`	Also look in dir for headers.
`-O2`	Optimize. Used for release builds, not while learning.

Building and running

Command	What it does
`make`	Build whatever is out of date.
`make clean`	Delete the built files, if the Makefile defines it.
`./program`	Run a program in the current directory.
`echo $?`	Show the exit code of the last command.
`touch file`	Update a file's timestamp, making make rebuild it.

> **NOTE**
>
> Two flags are worth adding to your habits permanently: `-Wall -Wextra`. They cost nothing and they turn a class of silent mistakes into loud ones.